A

Project Report On

GameCut AI

AI Assisted Video Editing and Highlight Generation Platform

Designed By:

Project Student

Submitted To:

Department of Computer Science

In the partial fulfillment of
B.Sc. Computer Science

Under Guidance of:

_____

Through:
THE HEAD,
DEPARTMENT OF COMPUTER SCIENCE

PROJECT REPORT
2025-26

DECLARATION

To,
The Head,
Department of Computer Science

Respected Sir/Madam,

I hereby declare that the project entitled “GameCut AI: AI Assisted Video Editing and Highlight Generation Platform” has been developed as original academic work. The analysis, design, implementation, testing, and documentation presented in this report are based on the current project source and its supporting documentation. External technologies and documentation used by the project are acknowledged in the references.

Date: _____
Place: _____

(Project Student)

CERTIFICATE

This is to certify that the project work entitled “GameCut AI: AI Assisted Video Editing and Highlight Generation Platform” has been satisfactorily carried out during the academic year 2025-26. The work demonstrates the use of web technologies, database management, Python media processing, artificial intelligence planning, FFmpeg rendering, and software testing.

Project Guide External Examiner Head of Department

Date: _____

ACKNOWLEDGEMENT

It is my great pleasure to express sincere thanks to all those who supported the completion of this project. I am grateful to the project guide and the Department of Computer Science for their guidance and encouragement. I also acknowledge the open-source communities behind PHP, Python, MySQL, FFmpeg, OpenCV, PySceneDetect, faster-whisper, and related tools used in the implementation.

Thank You.

Project Student

INDEX

Sr. No.	Chapter No.	Title	Page No.
1	I	Acknowledgement	
2	II	Synopsis of the Project	1-4
3	III	Main Report	5-43
4	IV	Annexure	44-47
5	V	Conclusion	48-49
6	VI	Future Enhancement	50-51
7	VII	References and Bibliography	52-53

Main Report includes Objective and Scope, Theoretical Background, Survey of Literature, Research Analysis and Design, System Planning, Hardware and Software, Life Cycle, Data Flow Diagram, ER Diagram, Activity Diagram, Class Diagram, Input and Output Screen Design, Process Involved, Testing Methodology, Test Cases, and Code of the Project.

CHAPTER II

SYNOPSIS OF THE PROJECT

GameCut AI - Problem Statement

Long gameplay recordings contain valuable moments, but creators must manually watch footage, identify exciting sections, trim clips, reframe them for mobile screens, add subtitles, and render a final video. This work is repetitive and difficult to scale. Existing basic editors provide manual tools but do not connect media analysis, highlight scoring, safe job orchestration, and output validation in one workflow.

- Manual review consumes time and attention.
- Short-form platforms require vertical output and clear captions.
- Long-running rendering must not block the browser.
- AI decisions must be validated before media commands execute.

GameCut AI - Objective and Scope

The objective is to develop a web-based AI-assisted video editing platform for gaming creators. The system accepts uploaded media, analyses speech and visual signals, selects candidate highlights, creates a constrained editing plan, renders a vertical short, and presents the output in a preview screen.

- Provide registration, login, upload, projects, processing, preview, and download.
- Use transcript, scene, audio, and visual analysis signals.
- Maintain durable job status, progress, retries, ownership, and events.
- Generate subtitles and thumbnails where the required data is available.
- Keep external publishing, custom model training, and cloud storage outside V1 scope.

GameCut AI - Process Description

The user uploads a gameplay video through the browser. PHP validates the request, stores the original file, creates project and video metadata, and creates a queued ai_jobs record. The Python worker claims the job, prepares an isolated workspace, analyses media, scores candidates, validates an editing plan, and renders the output with FFmpeg. The resulting file is validated with ffprobe and displayed on the Preview page.

GameCut AI - Expected Outcome

The expected outcome is a reproducible first-pass editing workflow. The system does not claim that every generated clip is perfect; instead, it provides a safe, explainable starting point that can be reviewed and improved. The design separates deterministic analysis from the planning layer so future intelligent providers can be added without changing storage, ownership, or rendering safety.

CHAPTER III

MAIN REPORT

Objective and Scope of the Project

GameCut AI is designed to reduce the effort required to transform long gameplay recordings into short-form video. It combines a browser interface with a backend service, a database, and an asynchronous Python worker. The main scope is creator productivity, not automatic publishing or a replacement for professional editing software.

- Automated first-pass highlight discovery
- Vertical 9:16 output
- Subtitle and thumbnail generation
- Durable job orchestration
- Preview and download

Theoretical Background

The project is grounded in multimedia processing, web application architecture, asynchronous job execution, speech recognition, scene detection, computer vision, and constrained planning. A video is treated as a time-based collection of streams. The system analyses the source without modifying it and creates derived artifacts in an isolated workspace.

Survey of Literature

Traditional video editing depends on manual timeline review. Cloud editing platforms add collaboration and managed rendering but may require external storage and service costs. AI highlight systems can reduce manual work but may be difficult to explain. GameCut AI adopts a transparent V1 approach: deterministic signals, structured plans, strict validation, and local/private storage.

- Manual editors: flexible but time-intensive.
- Cloud editors: convenient but dependent on network and service limits.
- AI editors: efficient but require explainability and safety boundaries.

Research Analysis and Design

The research analysis identifies the repeated tasks in short-form editing and translates them into software requirements. The design treats upload, analysis, planning, rendering, and preview as separate responsibilities. This allows each stage to be tested independently and prevents an invalid plan from reaching FFmpeg.

System Requirements Analysis

The system requires a browser, PHP runtime, MySQL database, Python environment, local media directories, and bundled FFmpeg/ffprobe binaries. Functional requirements cover authentication, upload, projects, jobs, analysis, planning, rendering, preview, download, regeneration, cancellation, and deletion.

Requirement	Description	Priority
User authentication	Register, login, sessions, logout	High
Media upload	Store source video and metadata	High
AI job	Queue and execute processing	High
Analysis	Transcript, scenes, audio, visual signals	High
Rendering	Vertical MP4, subtitles, thumbnail	High
Preview	Watch and download output	High
Safety	Ownership token and plan validation	High

Design of the System - System Architecture

GameCut AI uses a layered architecture. The presentation layer contains HTML, CSS, and JavaScript screens. The PHP application controls requests and database operations. MySQL stores user, project, video, job, and event state. The Python worker performs analysis and rendering. Private media storage holds originals, temporary artifacts, and processed outputs.

Database Design

The database separates user-facing summaries from canonical execution state. Projects and videos describe what the user sees. ai_jobs records the authoritative lifecycle of processing, while ai_job_events stores append-only history. This prevents progress and retries from being lost in a single mutable summary row.

Table	Purpose	Relationships
users	Registered accounts	One user has many projects and sessions
projects	Editing workspaces	Belongs to a user; contains videos
videos	Media metadata	Belongs to a project; may have output
ai_jobs	Execution state	Belongs to a video; has events
ai_job_events	Progress history	Many events belong to a job

Data Flow Diagram

The data flow begins with a user request and ends with a previewable media output. The interface sends requests to PHP. PHP reads and writes MySQL and controls private media storage. The worker reads job state and source media, performs analysis and rendering, and writes the processed result for the preview screen.

The main relationships are users to projects, projects to videos, videos to ai_jobs, and ai_jobs to ai_job_events. Sessions connect authenticated browser activity to users. The model keeps processing history available for operational review.

Activity Diagram

The activity begins with authentication and upload. The system validates the file, creates the project and job, and waits for the worker. The worker analyses, scores, plans, validates, renders, and validates output. The flow ends in success, failure, or cancellation.

Class and Module Design

The project is organized into PHP modules, JavaScript page modules, Python API routes, detection modules, agent modules, processing modules, and job modules. Each module has a focused responsibility. The agent layer calls safe Python functions and never executes arbitrary shell commands.

- php/bootstrap.php and php/helpers.php provide shared application support.
- python/detection contains scene, speech, audio, visual, and scoring modules.
- python/agents contains schemas, providers, tools, planner, and validator.
- python/processing contains preprocessing, rendering, subtitles, and basic editing.

Gantt Chart and System Planning

Project planning divides the work into preliminary investigation, requirements analysis, design, coding, testing, documentation, and final review. The current structure supports incremental delivery because each layer can be tested before the next layer depends on it.

Phase	Main work	Deliverable
Preliminary Investigation	Problem and user workflow	Project proposal
System Analysis	Requirements and constraints	Requirement list
System Design	Architecture, database, diagrams	Design document
System Coding	PHP, JS, Python, SQL	Working modules
Testing	Unit, integration, smoke tests	Test evidence
Documentation	Report and references	Final project report

Details of Hardware and Software Used

The project can be developed on a modern Windows computer with an Intel i5-class processor, 8 GB or more RAM, adequate storage for video files, PHP, MySQL, Python, a web browser, and bundled FFmpeg and ffprobe binaries. The application is designed for local development but can be deployed to a server after security hardening.

Category	Technology	Use
Frontend	HTML, CSS, JavaScript	Screens and browser interaction
Backend	PHP	Authentication, upload, APIs, project actions
Database	MySQL	Persistent application and job state
Processing	Python	Analysis, planning, rendering orchestration
Media	FFmpeg, ffprobe	Video processing and validation
Analysis	OpenCV, PySceneDetect, faster-whisper	Visual, scene, and speech signals

Detailed Life Cycle of the Project

The project life cycle includes initialization, requirements, planning, design, implementation, verification, evaluation, documentation, and deployment preparation. Each phase informs the next. Testing is not postponed until the end; the repository contains tests for the worker, API, scene detector, scoring, motion editor, rendering, and concurrency behavior.

AI Assisted Processing Pipeline

The pipeline uploads media, extracts audio, generates transcript segments, detects scenes, calculates audio and visual signals, scores candidates, creates an editing plan, validates it, renders a vertical short, burns subtitles, creates a thumbnail, and validates the final output.

Process Involved - Preprocessing

Preprocessing reads the uploaded source without modifying it. It prepares the analysis workspace and records media properties required by later stages. The workspace is isolated by job ID so concurrent jobs do not overwrite each other.

Process Involved - Candidate Selection

Candidate selection combines available transcript, scene, audio, and visual information. The scorer records reasons alongside candidates, making the result more understandable during debugging and review. Candidates are later constrained by duration and overlap rules.

Process Involved - Agent Planning

The V1 agent is an editing planner. Its structured output describes target duration, aspect ratio, clips, subtitle preference, thumbnail preference, and provider. The planner does not directly run FFmpeg, SQL, or arbitrary filesystem operations.

Process Involved - Plan Validation

Validation is performed before rendering. Invalid objects, unsupported ratios, empty clips, invalid timestamps, out-of-range values, excessive clip length, excessive total duration, and overlaps are rejected. This protects both correctness and the execution environment.

Process Involved - Rendering

The renderer trims clips, normalizes them, applies 1080x1920 vertical formatting with center crop, concatenates them, maps transcript timestamps, burns subtitles, creates a thumbnail, and checks the output with ffprobe. The database stores the relative output path after successful publication.

Process Involved - Job Ownership

A worker claim receives an execution token, lease, heartbeat, and isolated workspace. Progress, retry, failure, and publication updates require the current token. Regeneration creates a new revision and cancellation removes ownership safely.

Input Screen Design

The main inputs are registration details, login credentials, project title, source video selection, and actions to start, regenerate, cancel, or delete work. The browser provides usability feedback, while the server remains authoritative for validation and ownership.

Output Screen Design

The dashboard shows project summaries. The processing screen shows status and progress. The preview screen displays the completed video and download action. Project and settings screens provide history and configuration. The CSS structure keeps these page responsibilities separate.

Methodology Used for Testing

Testing combines unit testing, integration testing, smoke testing, and concurrency testing. High-risk boundaries are tested first: invalid timestamps, overlapping clips, stale workers, missing output, cancellation, regeneration, and failed processing.

Test Cases and Test Reports

Representative tests verify registration, upload, job creation, worker claims, plan rejection, vertical rendering, subtitle mapping, ownership guards, regeneration, and safe deletion. The repository includes dedicated Python tests and a PHP smoke test.

Test ID	Test case	Expected result	Result
TC01	Create user account	Account and session created	Pass
TC02	Upload source video	Metadata and original path stored	Pass
TC03	Reject overlapping clips	Plan rejected before FFmpeg	Pass
TC04	Claim queued job	Lease and token recorded	Pass
TC05	Render output	Output passes ffprobe	Pass
TC06	Stale publish attempt	Ownership guard rejects update	Pass
TC07	Regenerate project	New revision is created	Pass
TC08	Delete active project	Cancellation precedes cleanup	Pass

Code of the Project

The complete code is maintained in the GameCut-AI directory. Important areas include PHP routes and processing files, JavaScript page scripts, Python worker and API modules, detection and processing packages, database schema and migrations, and test suites. The report documents design and representative logic while keeping the source tree as the authoritative implementation.

Limitations of the Current System

The current version does not include dedicated game-specific event detection, custom model training, cloud storage, long-term memory, automatic publishing, or perfect semantic ranking. Subtitle accuracy depends on audio quality and recognition availability. These limitations define the future enhancement plan.

CHAPTER IV

ANNEXURE

List of Tables

This annexure lists the primary database entities, requirements, technology stack, test cases, and project modules used in the report.

List of Figures

Figures include system architecture, data flow diagram, entity relationship diagram, activity and processing flow, and the AI-assisted processing pipeline.

File and Folder Structure

The GameCut-AI folder contains PHP pages and helpers, CSS styles, JavaScript modules, API routes, database schema and migrations, Python agents, detection, processing, jobs, tests, bundled FFmpeg tools, uploads, and documentation.

Configuration and Deployment Notes

Environment values, database configuration, writable upload directories, PHP runtime, Python environment, and media binaries must be configured before local execution. Production deployment requires HTTPS, secure secrets, upload hardening, and monitoring.

CHAPTER V

CONCLUSION

Conclusion

GameCut AI successfully demonstrates an AI-assisted video editing workflow that combines a web application, database, durable job queue, Python media analysis, constrained planning, FFmpeg rendering, subtitle generation, preview, and export. The project turns a long source recording into a reviewable short-form result while preserving a transparent and testable architecture.

Project Contribution

The main contribution is the separation of analysis, planning, validation, rendering, and publication. This makes the system safer to operate and easier to extend. The durable ai_jobs record, ownership token, append-only events, and output validation provide stronger engineering foundations than a single synchronous upload script.

CHAPTER VI

FUTURE ENHANCEMENT

Future Enhancement

Future versions can add game-specific kill, goal, win, and event detection; multimodal semantic ranking; cloud storage; human-in-the-loop clip adjustment; more aspect ratios and branded templates; multilingual captions; social publishing integrations; and worker monitoring dashboards.

Future Work Plan

Future work should preserve the current safety boundaries while improving selection quality. The recommended order is stronger media metadata and event detectors, user review controls, scalable storage, provider integrations, production security, and operational analytics. Each enhancement should retain plan validation and ownership guards.

CHAPTER VII

REFERENCES AND BIBLIOGRAPHY

References and Bibliography

Project documentation and technical references used for the design and implementation include: GameCut AI docs/architecture.md; GameCut AI docs/ai-pipeline.md; database/database.sql; PHP documentation at php.net; Python documentation at docs.python.org; FFmpeg documentation at ffmpeg.org; OpenCV documentation at docs.opencv.org; PySceneDetect documentation at scenedetect.com; faster-whisper project documentation; and MySQL documentation at dev.mysql.com.

Final Bibliography Note

The project report uses the supplied academic report as a structural reference: cover, declaration, certificate, acknowledgement, index, synopsis, main report, annexure, conclusion, future enhancement, and references. All technical content and diagrams in this revised version describe GameCut AI only.